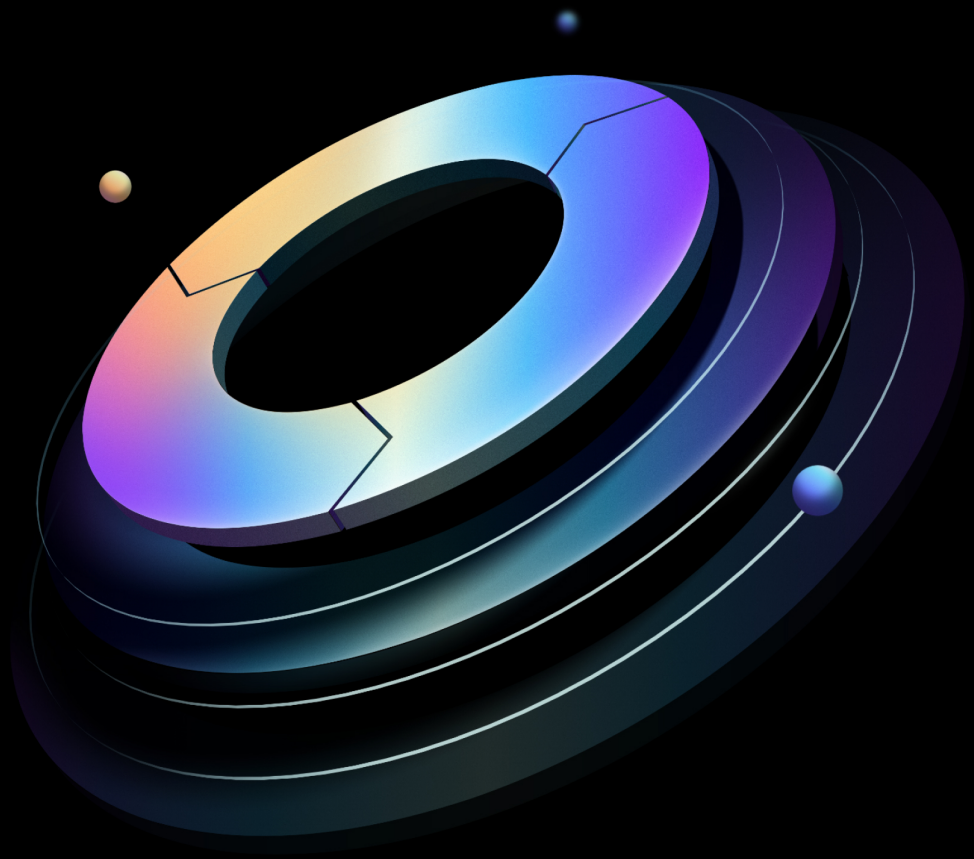
HashiCorp

# Boundary User Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

# Contents

**1 Introduction** **3**
1.1 Why use HashiCorp Validated Designs? . . . 3
1.2 Prerequisites . . . 3
1.3 Language and definitions . . . 4
1.4 Use cases covered . . . 7

**2 Transparent sessions** **8**
2.1 Purpose of guide . . . 8
2.1.1 Target audience . . . 8
2.2 Prerequisites . . . 8
2.2.1 Limitations . . . 9
2.3 Background and best practices . . . 9
2.3.1 Aliases . . . 9
2.3.2 Boundary Client Agent . . . 10
2.4 Validated architecture . . . 11
2.5 Workflow . . . 13
2.5.1 Checklist . . . 13
2.5.2 Platform operator . . . 13
2.5.3 Session users . . . 15
2.6 Conclusion . . . 19

**3 Credential management** **20**
3.1 Credential brokering . . . 20
3.2 Useful resources . . . 22

**4 Dynamic credentials** **22**
4.1 Leveraging dynamic credentials . . . 23
4.2 Credential injection . . . 23

**5 Session recording** **25**
5.1 Viewing recorded sessions . . . 26

**6 Just-in-time approval** **26**
6.1 Overview . . . 26
6.1.1 1. Enhanced security and compliance . . . 27
6.1.2 2. Operational efficiency . . . 27
6.1.3 3. Integration with approval workflow platforms . . . 27
6.1.4 4. Improved user experience . . . 28
6.2 Useful resources . . . 28

**7 Changelog** **28**
7.1 2026-08 . . . 28

**8 Credits** **28**

# 1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

## 1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

This User Guide covers the use cases that Boundary Enterprise supports once the platform is installed and operational. Use it to configure and enable Boundary features for your consumer teams, from foundational access workflows through to advanced, scaling-phase capabilities.

References to *Boundary* in this document apply to both HCP Boundary and Boundary Enterprise except where specified to indicate differences between the products. We also use *Boundary* in the context of the command-line tool where applicable.

## 1.2 Prerequisites

This guide assumes that Boundary Enterprise has already been deployed and brought into service. Before using this guide, complete the following:

- Deploy Boundary Enterprise following the Boundary Installation Guide.

- Review the Boundary Administration Guide for the system-level operations that support these use cases.

The use cases in this guide are ordered roughly from foundational to advanced, but you can navigate to any use case directly using the section headings.

### 1.3 Language and definitions

While this guide intentionally uses technology-agnostic language, there are some terms that do not translate seamlessly between providers. This document uses the following terms:

| Term | Definition |
|---|---|
| **Scope** | A permission boundary modeled as a container for resources. |
| **Global scope** | The top-level scope that encompasses all child scopes within the boundary system. It serves as the root of the hierarchical structure to organize resources. The resources such as storage buckets, storage policies, aliases, workers, users, groups, roles and auth methods can be configured at global scope level. |
| **Organ** | The intermediate scope level, also referred to as organizations, are a child scope of global. Identity and access management related resources such as users, groups, roles and auth methods can be configured at the organization and global scope levels. |
| **Project** | The lowest scope level and a child scope of organization. It allows logical grouping of resources within an organization, such as targets, host catalogs, credentials stores and sessions. |
| **Host catalog** | A collection of hosts that Boundary can connect to, organized into host sets. |
| **Host set** | A subset of hosts within a host catalog which are considered equivalent for the purposes of access control. |
| **Host** | A resource with a network address reachable from Boundary, such as a server or database. |

| Term | Definition |
| --- | --- |
| Target | A resource that ties network address information (via a direct address or by referencing host sets), credential libraries for injection or brokering (if desired) and port information to represent a networked service available for connection through Boundary. Targets also contain parameters, such as lifetime and connection count, to configure on the sessions created via authorization against the target.. A target can also be configured with ingress/egress worker filters that determine which workers are used to access targets. |
| Worke | A secure network proxy, enabling users to access private targets by establishing a direct network tunnel between the Boundary client on the user's machine and the target systems. |
| User | A resource that represents an individual person or entity for the purposes of access control. A user can be associated with zero or more accounts. A user authenticates to Boundary through an associated account and must be associated with at least one account before they can access Boundary. |
| Group | A resource that represents a collection of principals, allowing them to be treated equally for access control purposes. As a principal itself, a group can be assigned to roles, and any role assigned to a group is indirectly assigned to all users within the group. A group can be defined at the global, organization, or project scope levels. |
| Managed group | A resource that represents a collection of accounts. The collection is formed by evaluating account information (e.g. LDAP groups, OIDC claims) defined by the auth method's identity provider against the managed group's configuration. An account can be associated with zero or more managed groups within the same auth method. It can be used as a principal in roles. |
| Role | A role is a collection of permissions granted to any principal assigned to it. Users, groups, and managed groups can be configured as principals in a role. |
| Authentication method | The mechanism by which users authenticate to Boundary. Can also be integrated with external identity providers like LDAP, Active Directory, or OIDC providers. |
| Accou | A representation of a user's identity within a specific authentication method. Accounts can be created manually or automatically generated and are associated with a user in the same scope as the account's auth method.. |
| Credential | Authentication details such as passwords, tokens, or keys used to access resources. |

| Term | Definition |
| --- | --- |
| Crede store | Secure storage for managing and accessing credentials. This may be built-in to Boundary or contain information for accessing an external store like HashiCorp Vault. |
| Credential li- brary | A collection of credentials of the same type from a single credential store that can be brokered or injected into the network session when users are accessing the networked services via sessions. |
| Sessi | A session is a set of related connections between a user and a host. A session may include a set of credentials which define the permissions granted to the user on the host for the duration of the session. Limits can be placed on the session, such as a maximum lifetime and/or a maximum connection count. |
| Session record- ings | Recordings of user sessions for auditing and monitoring purposes. |
| Storag bucke | A container for storing session recordings and other data within Boundary. |
| Storage pol- icy | Rules and configurations governing the management and retention of data within storage buckets. |
| Availa zone (AZ) | A distinct data center within a region that provides redundant and isolated infrastructure to ensure high availability and failover protection. Each region consists of multiple AZs to offer resilience against system failures. |
| Region | A geographically distinct area that hosts multiple data centers, providing redundancy and fault tolerance for cloud services. Each region consists of multiple, isolated locations known as Availability Zones. |
| Instar | A physical or virtual server or hardware unit used for computing purposes. |
| Load bal- ancer | A hardware or software device used to distribute incoming network traffic across multiple servers. |

### 1.4 Use cases covered

This guide covers the Boundary features that consumer teams use once the platform is running:

| Use case | Summary |
| --- | --- |
| Transparent sessions | Seamless, passwordless access to authorized targets using aliases and the Boundary Client Agent. |
| Credential management | Accessing target credentials through Boundary credential brokering. Credential stores and libraries are configured by operators (see the Administration Guide). |
| Dynamic credentials | Using Vault-backed, ephemeral, time-bound credentials to access targets. The Boundary–Vault integration is configured by operators (see the Administration Guide). |
| Session recording | Viewing and playing back recorded sessions for compliance and incident investigation. Recording is enabled and managed by operators (see the Administration Guide). |
| Just-in-time approval | Requesting time-limited, on-demand access to targets through an approval workflow. |

Note

Operator tasks — identity provider integration, managed groups, scopes/roles/grants, audit logging, key rotation, target discovery, and worker topology — are covered in the Administration Guide and Installation Guide.

# 2 Transparent sessions

## 2.1 Purpose of guide

This architectural guide presents a pattern for implementing HashiCorp Boundary Transparent Sessions. It focuses on best practices for platform operators and session users to follow when deploying and using Boundary Transparent Sessions in a production environment.

The main benefits of following this approach:

- **Developer experience:** Enhance user experience by allowing quick access to authorized targets without requiring users to change existing workflows or tools.
- **Ensure compliance:** Enforce strong, consistent, identity-based security controls with least privilege access to meet regulatory and organizational policies and service level agreements.
- **Optimize security:** Minimize the risk associated with static ports and credentials.
- **Optimize costs:** Minimize time to delivery by optimizing end user workflows.

### 2.1.1 Target audience

This guide targets organizations adopting a passive connection session establishment. It is relevant to the following roles:

- **Platform operator**: Maintains and scales Boundary from an infrastructure perspective. This role may reside within security-focused teams or broader infrastructure platform groups.
- **Session user**: Connects to authorized targets quickly through sessions. This role may reside within development teams, security-focused teams, or broader infrastructure teams.

## 2.2 Prerequisites

To complete the steps outlined in this guide, you need the following:

- Boundary deployment
  - Aliases
  - Client Agent
  - Desktop Client (optional)

- Operating system
  - macOS or Windows

### 2.2.1 Limitations

Please refer to the list of Known Issues that may affect Boundary Transparent Sessions.

## 2.3 Background and best practices

Boundary is a secure access management solution for accessing resources in private environments. Boundary Transparent Sessions provides developers with a seamless working experience by eliminating the need to remember resource IDs or ephemeral ports to connect to assets. Boundary accomplishes this by working in the background to handle authentication, authorization and to transparently grant access to the selected target through DNS interception and rerouting traffic through a session automatically.

Transparent Sessions requires Aliases and Boundary Client Agent to be able to run.

### 2.3.1 Aliases

As a globally unique DNS-like string, aliases enhance the developer experience by allowing the end user to indicate which target they wish to connect to through an alias *value* that maps back to a target ID. The end user can now initiate a session through remembrance of a human-readable alias such as `boundary.dev`, `database.boundary` and `webserver.boundary`. HashiCorp recommends not using single-word aliases such as `database` or `webserver` as single-word aliases do not work intuitively for Windows. We recommend that aliases include a separator (.). Aliases like `database.boundary` work, but `database` does not work.

When users initiate a connection to an authorized target using an alias, Boundary generates a session and transparently proxies the connection on behalf of the user. Aliases are not independent resources, they are metadata linked to a target. While aliases may exist without a link to a target, if you attempt to use them without configuring a link to a target, they do not function as intended.

For full guidance on Target Alias creation, please visit our documentation.

### 2.3.2 Boundary Client Agent

Boundary Client Agent is Boundary's DNS daemon. When run alongside an authenticated Boundary Client, the agent establishes itself as the primary resolver on the client machine and intercepts DNS requests to reroute through a session. If a host name matches a Boundary alias that you have authorization to access, Boundary transparently proxies the connection on behalf of the user. In instances where there is no match to a Boundary alias, Boundary's Client Agent performs recursive queries to the previously set DNS resolvers. The default configuration of the Client Agent is suitable for most users.

#### Client Agent logging

You can find the Client Agent log locally in the following folder:

macOS

```
/Library/ApplicationSupport/HashiCorp/Boundary/boundary-client-agent.log
```

Windows

```
C:\Windows\Logs\boundary-client-agent.log
```

Example of establishing Client Agent session:

```
2025-05-08T19:16:15.942-05:00
2024-05-08T14:42:16.704-0700 [INFO]  1827615645: found matching Boundary target:
    query name=ssh.boundary.dev. query type=1 target id=ttcp_TZ1fHSiAac port=22
    client port=0 user id=503
2024-05-08T14:42:16.704-0700 [INFO]  1827615645: macos.(darwinIfaceImpl).AddIp: going
     to add address to interface: ip=100.68.11.191 interface-name=en0
2024-05-081T14:42:16.721-0700 [INFO]  1827615645: underlying listener: : addr
    =100.68.11.191:22
2024-05-08T14:42:16.721-0700 [INFO]  1827615645: adding cleanup at time:
    session_expiration="2024-04-01 05:42:16.679928 +0000 UTC" time until=7h59m59
    .958007s cleanup_time=28798
2024-05-08T14:42:16.721-0700 [INFO]  1827615645: matching qtype, continuing
2024-05-08T14:42:16.722-0700 [INFO]  1827615645: returning dns answer: qtype=1 ip
    =100.68.11.191
2024-05-08T14:42:16.724-0700 [INFO]  1827615645: starting proxy for : target name=ssh
    .boundary.dev. entry addr=100.68.11.191 listener addr=100.68.11.191:22
```

Example of Client Agent log showing user connection to an already established session:

```
2025-05-08T12:57:14.804-0700 [INFO]  User successfully authenticated: UserId=503
2024-05-08T12:57:24.950-0700 [INFO]  1256893734: handling request: question=ssh.
    boundary.dev. local-addr-net=udp local-addr=100.93.11.86:53
2024-05-08T12:57:24.957-0700 [INFO]  3664984977: handling request: question=ssh.
    boundary.dev. local-addr-net=tcp local-addr=100.93.11.86:53
2024-05-08T12:57:24.957-0700 [INFO]  3664984977: existing session found
2024-05-08T12:57:24.957-0700 [INFO]  3664984977: incoming info: qtype=1 local addr
    =100.93.11.86
2024-05-08T12:57:24.957-0700 [INFO]  3664984977: existing info: qtype=1 addr
    =100.68.11.191
2024-05-08T12:57:24.957-0700 [INFO]  3664984977: matching qtype, continuing
2024-05-08T12:57:24.957-0700 [INFO]  3664984977: returning dns answer: qtype=1 ip
    =100.68.11.191
```

## 2.4 Validated architecture

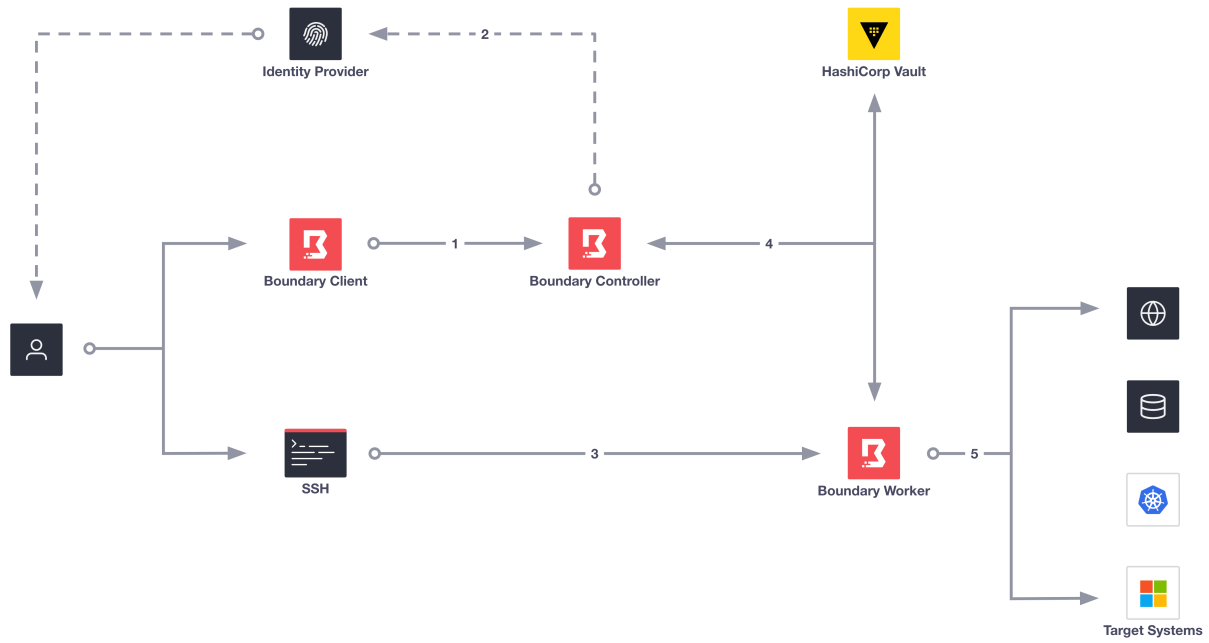



**Figure 1:** Boundary Workflow Overview diagram showing the 5-step process of user authentication, identity provider integration, client tool connection, credential retrieval, and session establishment

In the above diagram, Boundary works in the background to authenticate the user, authorize the user's login, and connect them to their target. Key steps, such as four and five, occur without end-user visibility, as Boundary brokers these steps machine to machine. We dive into one of the key components for Transparent Session in the next diagram.

1. User logs onto Boundary Controller using Boundary client.
2. If using an external Identity provider, Boundary redirects users to authenticate with an Identity Provider.
3. User uses preferred client tool (SSH terminal, browser, etc) to connect to Target.
4. Boundary controller retrieves credentials from Vault and sends them to the Boundary Worker.
5. Boundary establishes a connection and injects dynamic credentials into the session.

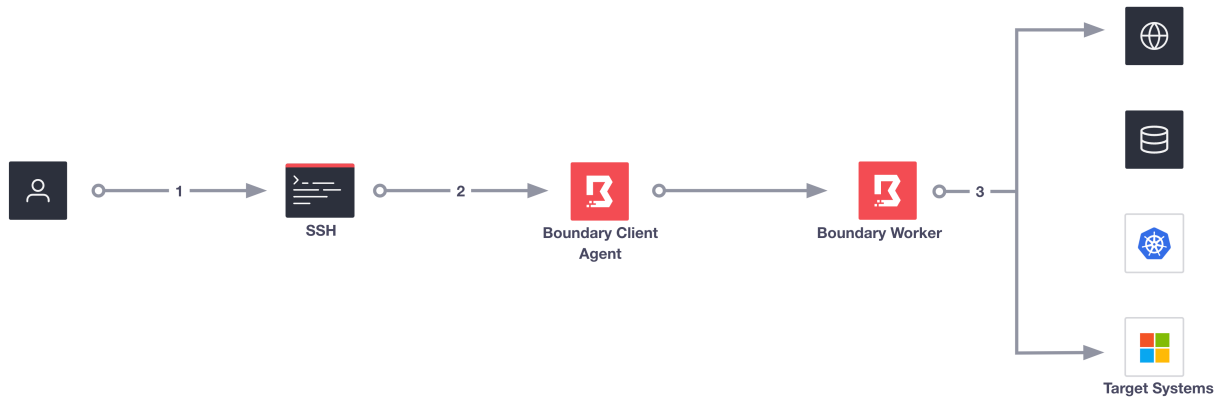



**Figure 2:** DNS Interception diagram illustrating how Boundary Client-Agent intercepts DNS requests and routes traffic through Boundary sessions automatically

In the above diagram, Boundary Client-Agent establishes itself as the primary resolver on the client machine and intercepts DNS requests made by the end user on the system. Once the client agent intercepts the DNS request it checks if Boundary manages the alias. If the alias does exist, and the end-user has authorization to connect to the target, the client agent reroutes traffic through a Boundary session automatically. In the above diagram, the user has already authenticated.

1. End-user uses preferred client tool (SSH terminal, browser, etc) to connect to the target.
2. Client agent intercepts the DNS request.
3. Client agent reroutes traffic through a session automatically.

## 2.5 Workflow

### 2.5.1 Checklist

In order for users to transparently access authorized targets, platform operators must ensure that the following is in place:

- Boundary installed on end user's machine:
  - macOS : Boundary Installer
  - Windows: Boundary Installer
- Configure Boundary (Controllers, Workers). If you have not configured Boundary yet, we recommend you follow our Boundary HVD before continuing.
- Enable IPv4 and IPv6 protocols.
- Uninstall any previous versions of the Boundary Desktop or CLI client in your local environment before you install the latest Boundary installer.
- For Windows workflows ensure target aliases use (.) within their naming convention. Single-name aliases do not work intuitively on Windows. For example, an alias of `mytarget` does not work but `mytarget.` does.

### 2.5.2 Platform operator

#### Configure aliases for targets

The Client Agent periodically requests an updated list of aliases from the controller. The alias might not work immediately after creation. Allowing a two-minute pause (or time delay) often resolves any connection issues. If connection issues persist after waiting two minutes, please visit our Client Agent troubleshooting guide.

You must enable both IPv4 and IPv6 protocols for your environment to ensure the Client Agent can start and perform DNS queries. The Client Agent requires access over UDP and TCP on port 53 for both IPv4 and IPv6.

If your cluster is running a version of Boundary less than 0.16.0, add the necessary grants for listing resolvable aliases. This ensures that the client agent can populate the local alias cache. Please review the grant section in our Boundary HVD.

As an example, you could add the grant: `type=user;actions=list-resolvable-aliases;ids=*`.

> **Note**
>
> Aliases are a global resource.

After authenticating to Boundary with the CLI or the Admin UI, create an alias during target creation:

Example aliases are database.boundary. webserver.boundary, or hostname.host-set.project.org.

```
boundary targets create ssh \
    -name 'Example Boundary SSH target' \
    -description 'This is an example ssh target' \
    -scope-id global \
    -with-alias-authorize-session-host-id hst_1234567890 \
    -with-alias-scope-id global \
    -with-alias-value example.alias.boundary
```

Alternatively, if the target already exists, you can create an alias for an existing target with:

```
boundary aliases create target \
    -name 'Example Boundary alias' \
    -description 'This is an example alias for target tcp_1234567890' \
    -scope-id global \
    -destination-id tcp_1234567890 \
    -value example.alias.boundary \
    -authorize-session-host-id hst_1234567890
```

Parameter `authorize-session-host-id` specifies a particular host ID when authorizing a session. This becomes vitally important when you want to direct a session to a specific host within a target's host set. Typically Boundary selects a host at random from the set for a session. When using `authorize-session-host-id` you override this behavior and explicitly choose which host to connect to.

Once the target and alias exist, you can connect to the target via the alias using any client tool:

```
ssh example.alias.boundary
```

So long as you have authorization to access the target, Boundary transparently proxies the connection of the end user in the background.

### 2.5.3 Session users

For session users, using Boundary Transparent Sessions elevates the developer experience as they do not need to know IP addresses, port numbers of their target. They do not need to change their existing workflows or preferred client tools. They have little to zero interaction with the Boundary client tools. Lastly, when you enable credential injection, the user gets a completely password-less experience.

> Note
>
> For the following workflows, if you know the alias assigned to the target, after authentication, you can generally skip to step four.

#### Confirm the Boundary Client Agent

Next, confirm that you have installed Boundary Client Agent

```
boundary client-agent status
```

#### Secure Shell Protocol sessions

1. Authenticate to Boundary
2. Open Boundary Desktop or CLI client and find the target alias
3. Connect to the target using SSH

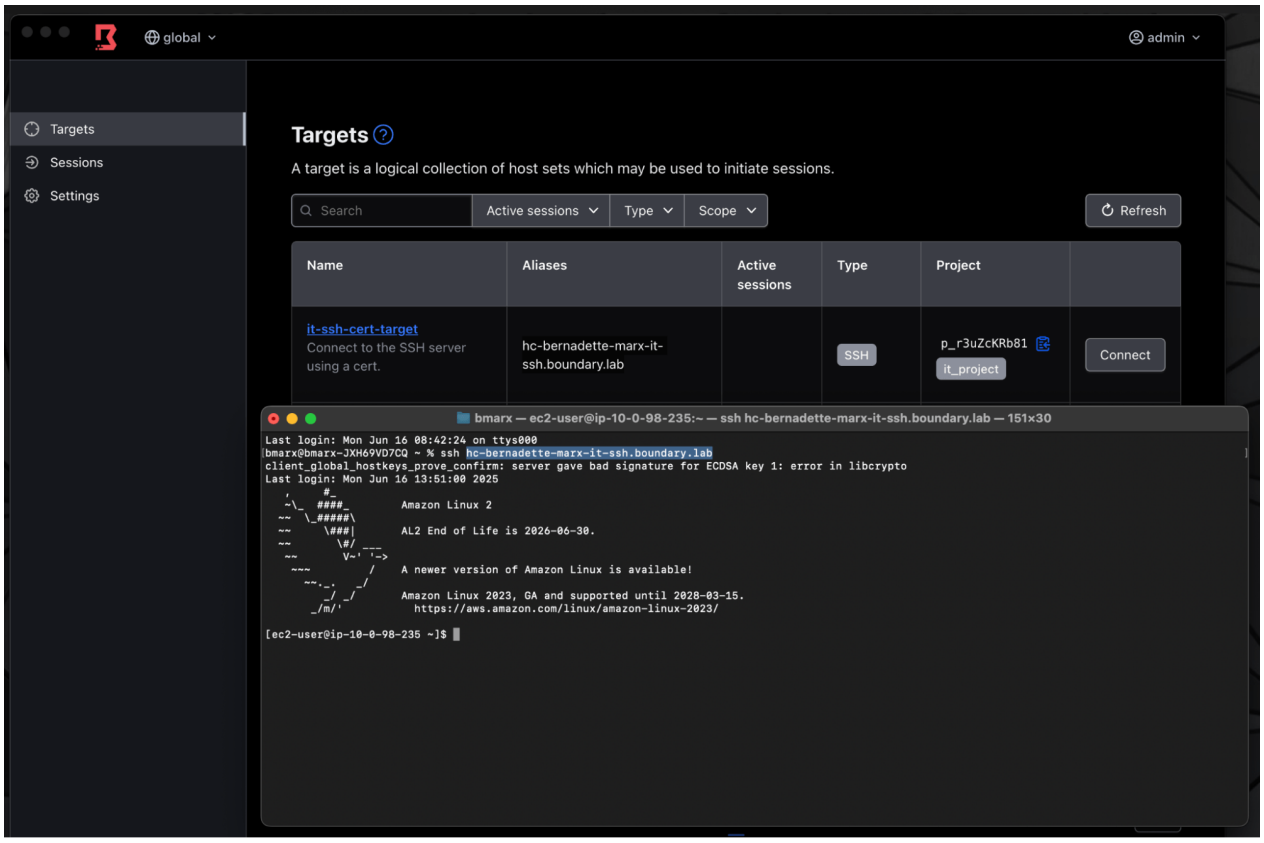



**Figure 3:** Screenshot showing SSH access to a target through Boundary using a target alias instead of direct IP connection

Access the target through SSH with target alias.

### Remote Desktop Protocol sessions

1. Authenticate to Boundary
2. Open Boundary Desktop or CLI client and find the target alias
3. Connect to the target using RDP

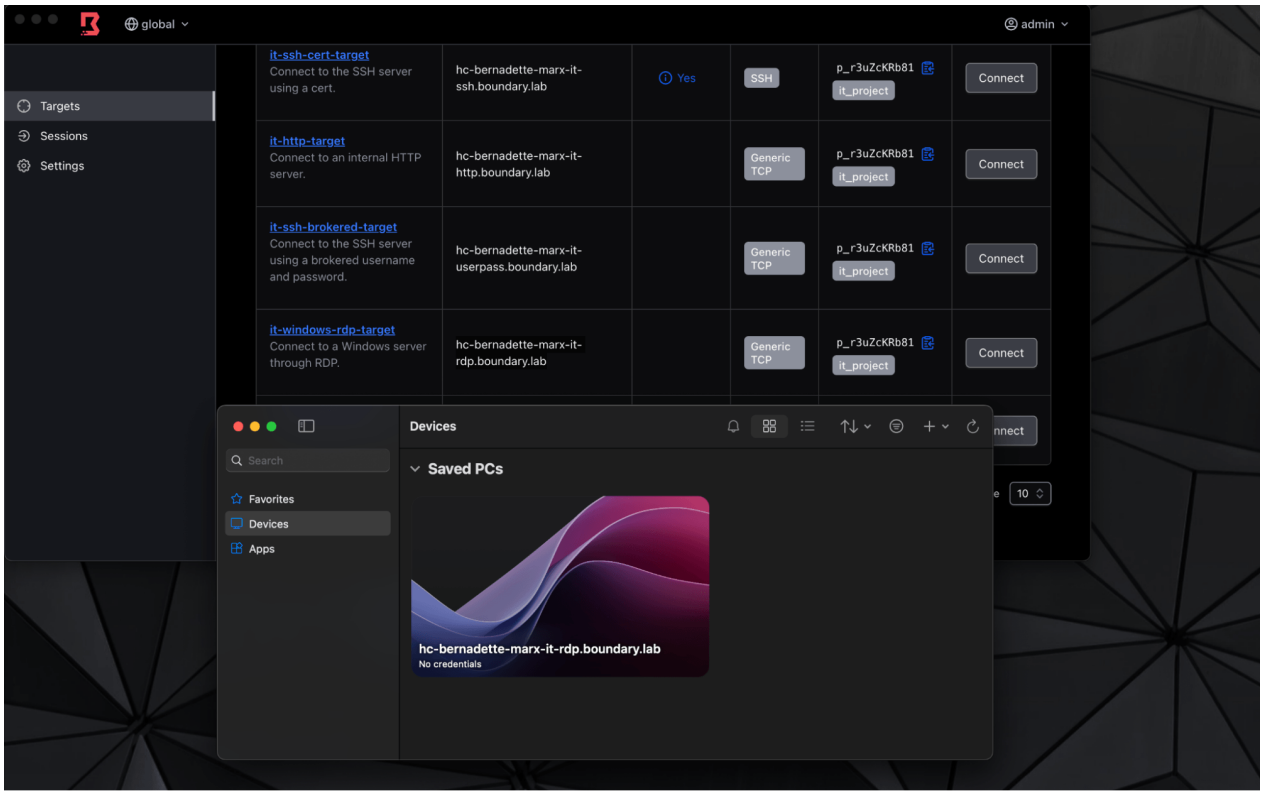



**Figure 4:** Screenshot of Remote Desktop Protocol connection through Boundary using a target alias

Access the target through RDP with target alias.

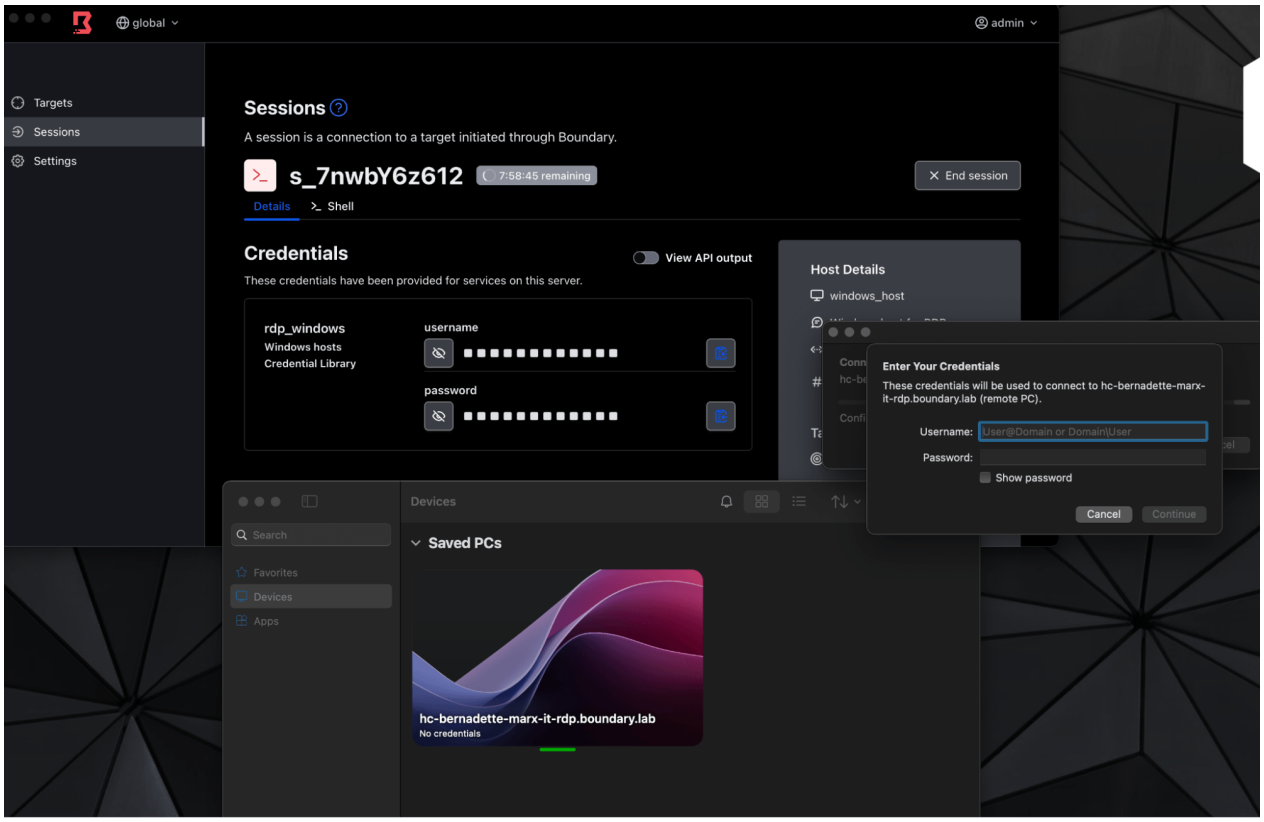



**Figure 5:** Screenshot showing credential entry dialog for RDP connection with credentials provided by Boundary through notification prompts

Enter credentials for the connection, provided by Boundary (prompted for redirect through notifications).

### Web access

1. Authenticate to Boundary
2. Open Boundary Desktop or CLI client and find the target alias
3. Enter alias into browser
    - If the certificate's Subject Alternative Name uses wildcards or matches the alias, SSL/TLS remains intact

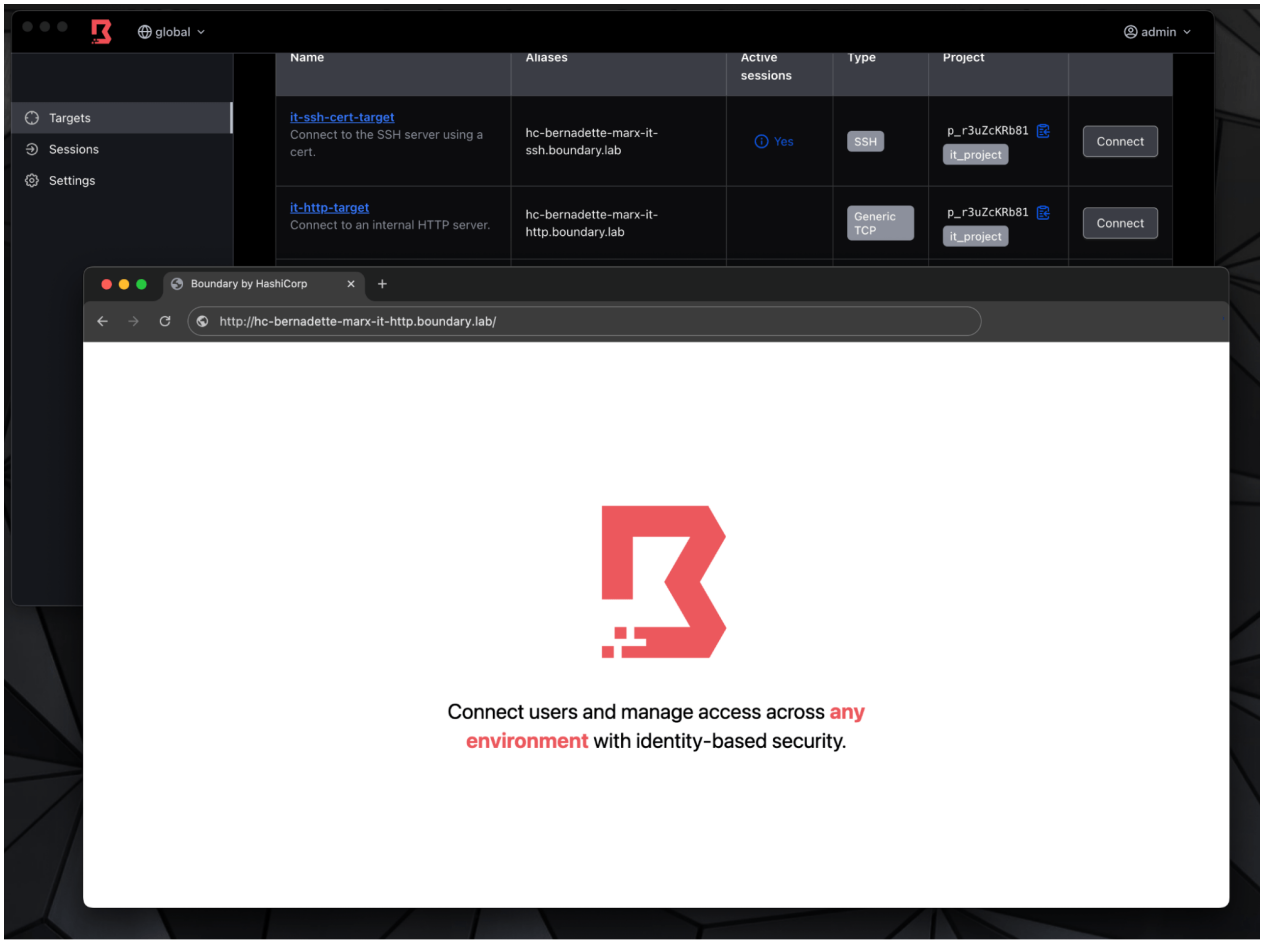



**Figure 6:** Screenshot demonstrating HTTP target access by entering http:// followed by the alias target name in a web browser

Access the target through HTTP by adding http:// in front of the alias target name.

## 2.6 Conclusion

The above guidance shows how to configure and connect to targets with Boundary's Transparent Sessions on macOS and Windows. Transparent Sessions offers a seamless experience for developers and non-technical business users.

Refer to our Boundary documentation to learn the known issues before deploying in production.

To learn more about using Transparent Sessions, check out the following resources:

- Transparent Sessions developer documentation
- Configuring Transparent Sessions
- Boundary Client Agent
- Boundary Aliases

# 3 Credential management

Boundary presents credentials to you through credential brokering. Credential stores and libraries are configured by platform operators — see the Administration Guide: Credential store management.

## 3.1 Credential brokering

Credential brokering is a workflow, where Boundary retrieves credentials from a credentials store and presents them to the end user. The end user then enters the credentials into the session when prompted.

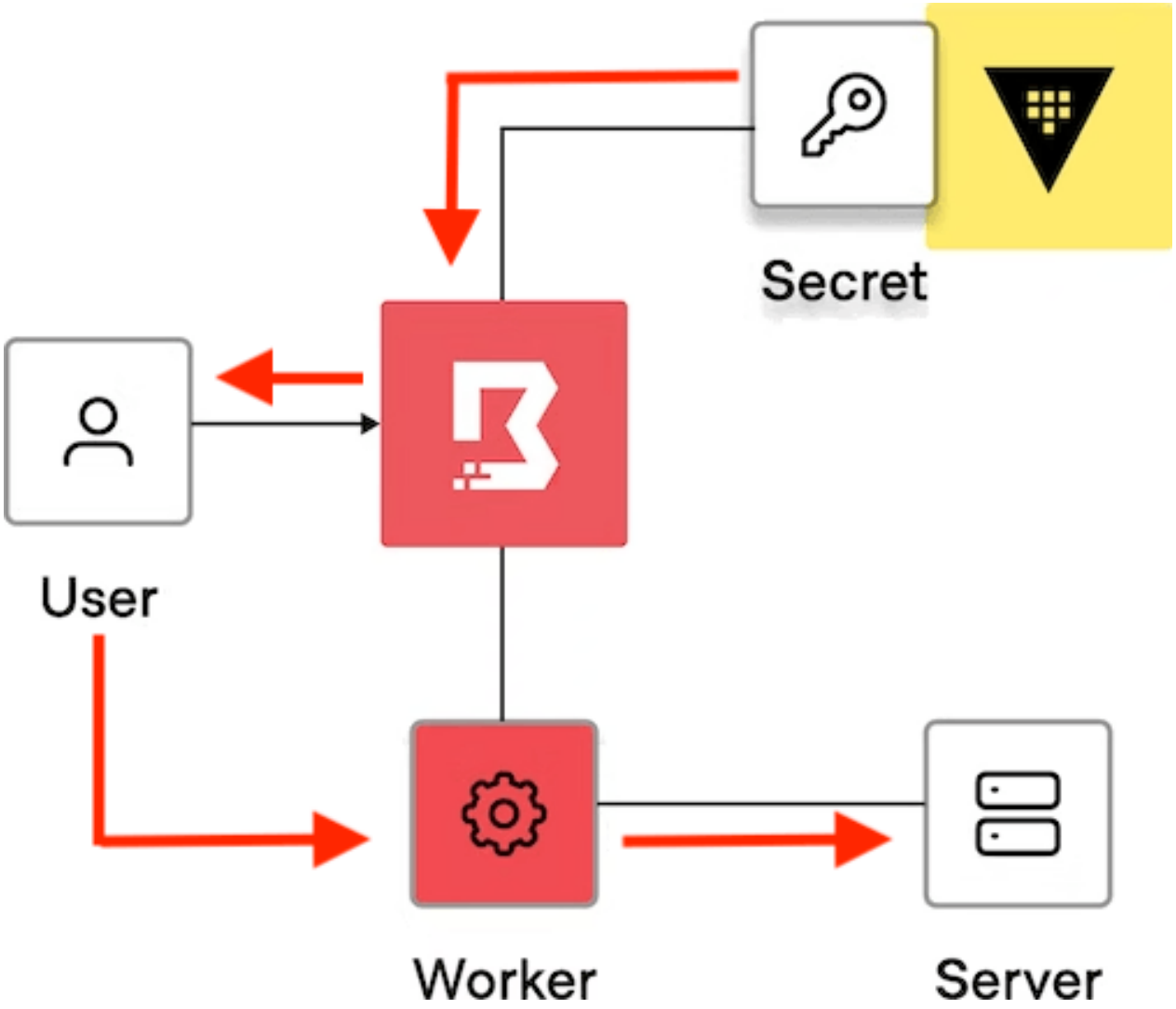



**Figure 7:** Boundary Credential Brokering

You can attach brokered credentials to either TCP or SSH targets. Brokered credentials can take the form of a token, username, and password, SSH private key, certificate, JSON blob, or an unstructured secret stored in Vault, for example.

You can find how to configure targets with credential brokering following this guide.

### 3.2 Useful resources

The following resources help you to implement, troubleshoot, and resolve credential brokering related issues.

- Credential Management Docs
- Credential Management Tutorials

## 4 Dynamic credentials

The Boundary–Vault integration that powers dynamic credentials is configured by platform operators — see the Administration Guide: Vault integration for dynamic credentials.

One of the key features of Boundary is its ability to broker or inject dynamic credentials through integration with an external credential management system - HashiCorp Vault. This capability is facilitated by dynamic credential stores, which provide on-demand, ephemeral credentials to users. Dynamic credential stores in Boundary enable the secure generation and management of temporary, time-bound credentials that are used to access various targets like databases, SSH servers, and other services. Unlike static credentials that are long-lived and manually managed, dynamic credentials are created just in time and automatically expire after a specified period, reducing the risk of credential leakage and minimizing the window of opportunity for unauthorized access.

The integration between Boundary and Vault improves two main areas of concern for organizations:

- Security posture for remote access
- Workflow efficiency

Integrating Boundary and Vault achieves these goals by enabling end-users to access targets without needing to manually distribute credentials.

Organizations should configure credentials to be dynamic and ephemeral by attaching a specific time to live (TTL) to the credential. This provides the highest level of security by applying a finite amount of time for those credentials to be used.

Timely access to resources creates improvements in workflow efficiency by removing manual approvals for access in favor of highly scoped, preconfigured access requests. Additional end-workflow improvements are added by removing credential management from the end user.

When integrated with Vault, Boundary has to be assigned a periodic, renewable, orphan token from Vault. Each credential store needs a separate Vault token.

The following have no impact on Vault's client count:

- The number of Boundary targets that source credentials from the stores
- The number of users connecting to the targets
- The number of sessions that get created
- The number of credential libraries the credential store contains

### 4.1 Leveraging dynamic credentials

End users have three workflows that can be operationalised for connecting to a target:

1. **Traditional Authentication** - when an end user connects to a target, Boundary initiates the session, but the end user must know the credentials to authenticate into the session. This workflow is available for testing purposes, but it is not recommended because it places the burden on the users to securely store and manage credentials.
2. **Credential Brokering** - credentials are retrieved from a credentials store and returned back to the end user. The end user then enters the credentials into the session when prompted by the target. This workflow is more secure than the first workflow since credentials are centrally managed through Boundary. For more information, see the credential brokering concepts page.
3. **Credential Injection (Recommended)** - credentials are retrieved from a credential store and injected directly into the session on behalf of the end user. This workflow is the most secure because credentials are not exposed to the end user, reducing the chances of a leaked credential. This workflow is also more streamlined as the user goes through a passwordless experience. For more information, see the credential injection concepts page.

### 4.2 Credential injection

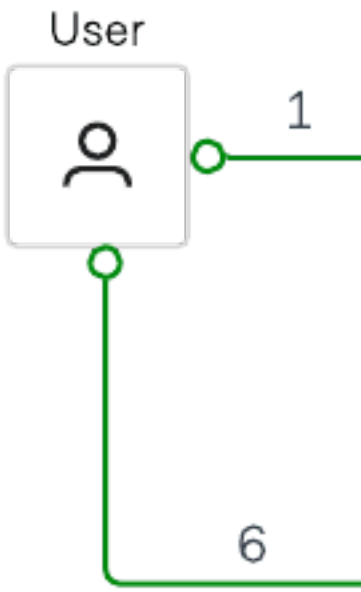
Credential injection is the process by which a credential is fetched from a credential store and then passed on to a worker for authentication to a remote machine. With credential injection, the user never sees the credential required to authenticate to the target. This provides a passwordless experience for the user, as the worker does both session establishment and authentication to the

target on behalf of the user. This process differs from credential brokering, where credentials are returned to the user rather than injected into the session on worker nodes.

Consider a scenario where a user wants to access a target using SSH as shown in the diagram below. In this scenario, the user must authenticate to Boundary and must be authorized to access the remote host/ server. Once authorized, a credential is generated for the particular session and is injected directly into the session. This allows the user to establish a remote session with the target.

Credential injection works as follows:

1. A user initiates a session to connect to a remote target by authenticating to Boundary and choosing the target.
2. The Boundary controller requests a dynamic SSH credential to be generated by Vault for this particular session. Note that for private Vault, a self-managed HCP worker will help in the connectivity between the Boundary control plane and private Vault cluster.
3. The Boundary controller provides those credentials back to the Boundary worker
4. The Boundary worker then passes the credential to the target and authtenicates on behalf of the user
5. The Boundary worker authenticates on behalf of the user. In this workflow, the user/client never has access to the credential.




6. Once the credentials have been authenticated, a user session can be initiated.

Credential Injection supports both Static and Dynamic Credential stores. For further details on credential injection and associated security considerations, reference credential injection

## 5 Session recording

Session recording is enabled and managed by platform operators — see the Administration Guide: Session recording administration.

For reasons similar to Audit Logs, an essential principle of securing access to sensitive resources

is creating a record of users' access and actions over remote sessions. In this context, recordings of remote access are often necessary.

Many organizations require the ability to record and playback sessions as an auditing capability for compliance and threat management.

In Boundary, a session represents a set of connections between a user and a host from a target. A session begins when an authorized user requests access to a target and ends when the access is terminated.

When you enable session recording on a target, any user session that connects to the target is automatically recorded. An administrator can later view the recordings to investigate security issues, review system activity, or perform regular assessments of security policies and procedures.

### 5.1 Viewing recorded sessions

Allowed users can view and play back recorded sessions through the Boundary administrative web portal. Recordings let you review user actions and investigate incidents. Refer to find and view recorded sessions for details.

## 6 Just-in-time approval

The approval-workflow platform integration is configured by platform operators — see the Administration Guide: Approval workflow integration.

### 6.1 Overview

Enterprises are increasingly looking to improve their enterprise digital workflows by integrating Just-in-Time (JIT) approval workflows with HashiCorp Boundary to enhance their security, compliance, and operational efficiency.

For example, a site reliability engineer might need higher permissions to fix an issue on sensitive systems, or an external contractor could need temporary access to an application. By integrating with approval workflow tools like PagerDuty, ServiceNow, and Slack, enterprises can enable just-in-time requests and approvals for time-limited access. This ensures that privileged roles are only active when a request is made and approved.

Here are key reasons why Enterprises integrate just-in-time approval workflow with Boundary:

#### 6.1.1 1. Enhanced security and compliance

- **Minimized Access Windows:** JIT approval workflows ensure that access is granted only when needed and for a limited duration. This reduces the risk of unauthorized access and potential security breaches.
- **Audit and compliance:** Integrating JIT workflows allows for detailed logging and the context of who accessed what purpose and when, which is crucial for compliance with industry regulations and internal security audit policies.

#### 6.1.2 2. Operational efficiency

- **On-Demand Access:** With JIT workflows, platform teams, partners and contractors can request access as needed, eliminating the need for standing permissions that might not be necessary at all times.
- **Streamlined Approvals:** Integrating with digital workflow platforms like PagerDuty, ServiceNow, and Slack enables seamless and rapid approval processes. Approvals can be managed directly within the tools that teams are already using, reducing friction and speeding up operations.

#### 6.1.3 3. Integration with approval workflow platforms

- **PagerDuty:** Integrating with PagerDuty allows for automated incident response workflows. When an incident occurs, necessary personnel can request and receive access to critical systems immediately, ensuring a quick and efficient resolution.
- **ServiceNow:** Service Now is widely used for IT service management. Integrating JIT workflows with ServiceNow allows for access requests to be part of existing ITSM processes, enhancing visibility and control.
- **Slack:** Slack integration allows for real-time communication and approval. Teams can manage and approve access requests directly within their Slack channels, leveraging the collaboration platform for immediate action.

#### 6.1.4 4. Improved user experience

- **Simplified Access Requests:** Users can request access through their existing approval workflow platforms, reducing the learning curve and making the process more intuitive.
- **Real-Time Notifications:** Integration with these platforms ensures that approvers are notified in real-time, enabling faster responses and reducing wait times for users.

### 6.2 Useful resources

- HashiConf 2024: Building a break glass solution with HashiCorp Boundary + Vault and ServiceNow integration
- HashiCorp Blog: Just-in-time approval workflow with Boundary and Azure

## 7 Changelog

This page records notable changes to the Boundary User Guide.

### 7.1 2026-08

- Restructured legacy Boundary HVD content into the HVD 2.0 Boundary User Guide as part of the Installation / Administration / User guide migration.

## 8 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Andrei Burd
- Danny Knights
- Jeff Mitchell

- Jim Lambert
- Mark Lewis
- Nick Wong
- Ravi Panchal
- Sai Linn Thu
- Shriram Rajaraman
- Theodore Marx
- Tony Phan
- Van Phan